데모 영상까지

영상

러나 강추 —
트 살아있는
시작

발표는 영상으로 —
NG 나면 다시 찍으면 된다.
라이브보다 유리

안이 그 방향이다.

전원 완료

스킬 · ponytail

막는다

분 영상으로 증명

전체 공유

e git ignore · 저장소에 올리지 않을 파일 목록(.env 등 보호)



/
다

로 —

자동 검토 서비스)

코드가 평준화되면, 차별화는 **도메인 지식과 고객 이해**에서 나온다 — 창업팀인 **여러분의 경쟁 우위**

m/blog 해커톤 수상자 발표

— 두 갈래로 갈린다

거의 맞음

개발자 불만 1위 · 66%

검증 없이 통과

↓

새벽에 장애 ⚠

원인 찾기 몇 배 어려움

한 단위. 확인 없이 다음 기능으로 넘어가지 않는다 — 오늘

```
ght, but not quite"
```



합니다

린 방법

x 1개

나

, 개인 결제 불필요)

여기서

ub.com 무료 가입)

공유 자원. 닿으면 5시간 후 리셋(그때가 수면·회의 타이밍) ·

권고

설

r · VS Code

클로드 앱

드를 보며 같은 엔진

그대로, Code 탭에서 폴더 지정

북은 터미널, 나머지는 팀원 취향대로.

: version control · 코드 변경 이력을 저장·복원하는 도구

● WINDOWS — 시작 → "PowerShell" → Enter

```
> irm https://claude.ai/install.ps1 |
iex
```

iex

히 종료(⌘Q)했다가 다시 열기 — 재시작해야 claude 명령을

cm.com 설치(전부 Next) 후 PowerShell 재시작

을 저장·복원하는 도구

는 각자 계정 → 승인 → 터미널 복귀

공

데스크톱 앱

cmux
여러 에이전트를 동시에 병렬 실행 · macOS 네이티브 터미널
(Mac 전용)

완전 종료 → 다시 열기 — 원인의 90%. Mac은 ⌘Q까지

V · Windows **우클릭** 또는 Ctrl+V · 안 되면 직접 타이핑

에 표시된 **URL을 복사**해서 브라우저 주소창에 직접 붙여넣기

n.com → Git for Windows 설치(전부 Next) → PowerShell

멘토 호출 — 붙잡고 있는 시간이 제일 비싸다

장·복원하는 도구

커톤 기간의 작업장)

e

dex.html **한 파일**로 만들어줘. 제목, 한 줄 소개, 핵심 기능
줘.

내 서비스가 표시되면 — **완료**

면 끝

다 **대화 초기화** —

스크린샷 붙여넣기

말 대신 이미지 — 캡처 후 **창에 드래그**

도가 떨어진다

· 붙여넣기는 **Ctrl+V (Mac도!)**

는 /plan) — 큰

"되돌려줘"

"마지막 잘 되던 상태로 되돌려줘" — git 커밋 기반 복원 기능

.md 자동 생성 — 새 프로젝트에서 1회

을 시작할 때

을 유지한 채 같은 작업을 이어갈 때

현황 — 가득 차기 전 /compact·/clear 판단 근거

정 잔여 한도 관리 (팀 공유 자원)

확인을 생략하는 **시작 옵션**. 빠르지만 위험하다 — 기본은
elector → Bypass)

권고

억

오를수록 느려지고 부정확해진다 — 밤샘처럼 긴 대화일수록.

640k / 1M 토큰 · **64%**

을 차지

엉뚱한 수정 · 반복

— 맥락 유지,

/clear

완전 초기화 — 새 기능 시작할 때

지면 /context로 점검

진다

팀원

권고

에이전트 = 병렬 팀원

에이전트 · 조사

aude
를 지휘

→
작업을
병렬로 분배

에이전트 · 구현

에이전트 · 검토

여러 개가 동시에 진행

를 동시에 — 설치도 호출도 "superpowers 설치해줘"
디로.

년 알아서

해줘"
rketplace
ketplace

면

조사 절차부터 —
어든다

③ 끝났다고 하면

"다 됐습니다" 전에 **스스로 검증** —
검증 규칙을 AI에게도

로 만들자"로 속도 조절.

다

권고

ail

스킬 — 템플릿 같지 않은 고유한 UI를 설계

인 방지 스킬

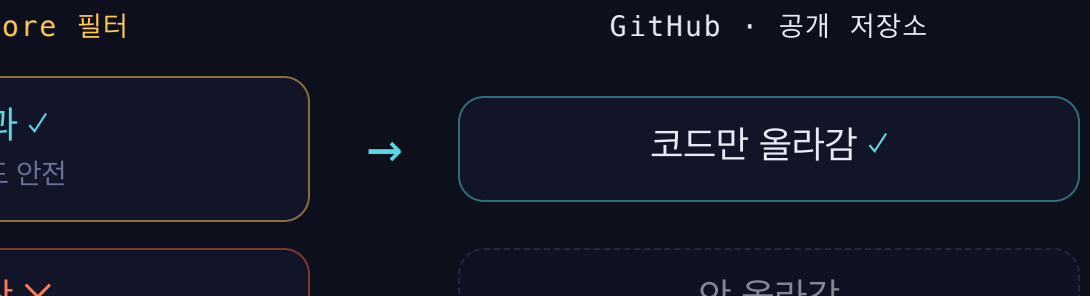
인 명세 — 골라서 프로젝트 폴더에

갤러리 — 링크 붙여 "이 느낌으로"

니어" 플러그인 — 불필요한 코드 증가를 막는다

지는 디자인 담당이 보고 팀에 맞는 것만 — 설치 커맨드는 공유 자료에.

은 .gitignore



e가 막음

⊘

로컬에만 남음

그대로 → **GitHub에 키 노출** → 봇이 몇 분 안에 도용

v로 옮기고, **.gitignore에 .env가 등록**돼 있는지 확인해줘. 데모

e git ignore · 저장소에 올리지 않을 파일 목록(.env 등 보호)

고,
명한다

시간은 행사 공지 기준

권고

— 5단계

심 기능 3개와 만들지 않을 것 목록을 PLAN.md로 확정

면이 비어 있어도 좋다 — 흐름 1개가 실제 URL에서 입력→결과까지 동작

람은 교대로 휴식, 작업은 계속 · 여러 명이 동시에? → 워크트리로 병렬(부록)

영에 쓸 데이터까지 미리 입력 — 빈 화면으로 찍지 않는다

기능 금지 — 버그 수정 · 영상 촬영 · 제출만

동작 — 배포를 미루는 팀이 가장 많이 실패한다

반드시 문제가 생긴다

ator · 웹 주소

권고

함정은 치환

함정 기능 → 치환

× 로그인 → **데모 계정 2개 하드코딩**

× 실시간 채팅 → **"문의 보내기" 폼**

- × 결제 → **가짜 버튼 + 완료 화면**
- × 푸시 알림 → **화면 안 알림 목록**
- × 관리자 페이지 → **안 만든다**

, 이번 해커톤엔 없는 기능. 심사위원은 로그인 화면에 점수를

권고

록한다

명세
로 말한다

복

번에
하나

2 · 생성
AI가 만든다

검증
러본다

, 하나씩 시키면 **100%짜리 3개**. 끝나면 **커밋 + /clear** → 다음

작 · 노트북이 죽어도 **서비스는 산다**

— 나머지는 Claude가 자동으로

Vercel
자동 빌드

→
공개

살아있는 URL
myapp.vercel.app

"베이트해줘" 한마디 · URL은 팀방 공유 → 폰에서 접속 테스트

6시간 룰의 이유. 배포 직전, **보안 점검 프롬프트** 한 번(앞 장).

장·복원하는 도구

권고

2:10–2:40

진짜 URL

어디까지 갈 것인가

가장 흔한 패배 패턴

n **Win+G + Clipchamp** · 해피 패스만, 2테이크

" · 상세 체크는 부록·메모에

tor · 웹 주소

첫 페이지가 나왔다

서

막는다

분 영상으로 증명

e git ignore · 저장소에 올리지 않을 파일 목록(.env 등 보호)

합니다 🚀

재현 (오늘 한 것)

r HO-2에서)

함

장 네트워크 변수 제거

Win Win+G)

CLAUDE.md

문제는 "___".

과 함께 확정하고 저장

목록 · 작업 순서

새 도구는 내게 물어보기

해

만들 기능과 범위를 미리 정리해 두는 파일

제약 + 완료기준

EXAMPLE

[맥락] 메인 페이지에 산책 요청 목록이 있어.

[목표] 상단에 "요청 등록" 폼 추가 — 날짜·시간대·메모 입력, 저장하면 목록 맨 위에 바로 표시.
[제약] localStorage 사용, 기존 목록 디자인 유지.
[완료기준] 등록 → 목록 반영 → **새로고침해도 유지**.

기능이 끝났다

화면을 더 신뢰감
크기만 다듬고

"방금 기능을 **내가 직접 확인할 방법**을
단계별로 알려줘 — 순서대로 눌러볼게"

내 손으로.

권고

순서대로

복사·붙여넣기 + "원인부터 설명하고 고쳐줘"

샷 붙여넣기 — 이미지가 천 마디 말보다 낫다

`/clear` → **더 작은 단위로** 다시 요청

잘 되던 상태로 되돌리고, 다른 방법으로 접근해줘"

을 **우회(함정 치환)**하거나 **멘토 호출**

있다

권고

나절을 붓는다

톤에선 익숙하고 검증된 스택만

구조 변경 금지

· .env와 .gitignore로

e git ignore · 저장소에 올리지 않을 파일 목록(.env 등 보호)

다 = 한 세트

나씩

위험

범위

드시 실패한다

e git ignore · 저장소에 올리지 않을 파일 목록(.env 등 보호)

심화 · 선택

병렬

개로 펼쳐 각자 다른 기능을 **동시에** (사람도, AI 🤖도 폴더 하나씩)

· 👤 나 · 로그인

🤖 AI · 결제

→
다 되면

📁 **main** ✓
완성본 하나로

합치기(병합)

합쳐서 완성

AI · 채팅

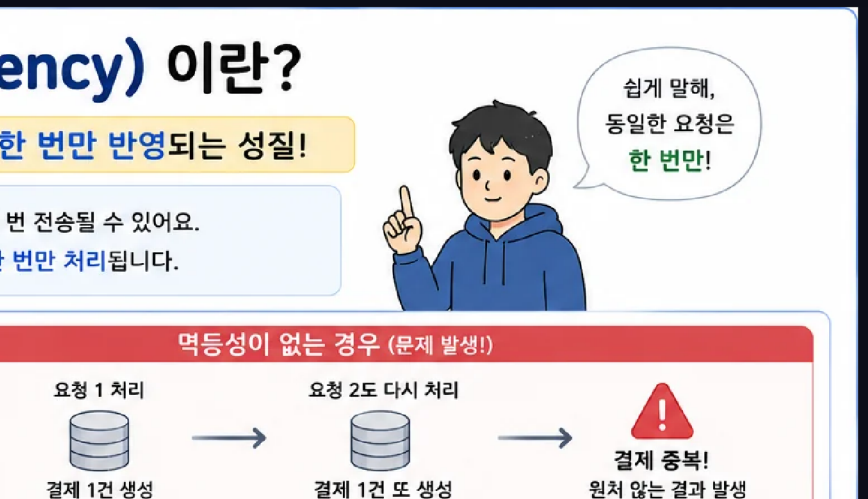
러 개 · 각자 **동시에**, 서로 안 부딪힘

나 더

토리 안에선 여전히 "한 번에 하나".

통과 후 권장

개념 · 참고



멱등성이 있는 경우 (안전!)

요청 1 처리 → 요청 2 도착 → 중복 처리 X

결제 1건 생성 / 이미 처리된 요청인지 확인 / 결과는 한 번만!

어떻게 멱등성을 구현할까요?

1. 고유한 키 발급
   요청마다 고유한 멱등성 키를 함께 전송
   Idempotency-Key
2. 요청 처리 및 저장
   요청을 처리하고, 결과를 키와 함께 저장
3. 중복 요청 확인
   같은 키로 요청이 오면 저장된 결과를 그대로 반환

같은 키로 들어온 요청은 새로 처리하지 않고, 기존 결과를 응답합니다.

→ **안전하고 정확한 서비스**를 만들기 위한 기본 원칙입니다.

개념 · 참고

ACID의 4가지 원칙

A Atomicity (원자성)
모든 작업이 성공하거나, 하나라도 실패하면 전체가 취소돼요.
모두 성공 / 모두 실패

C Consistency (일관성)
작업 전후에 데이터가 항상 올바른 상태를 유지해요.
올바른 상태 → 올바른 상태

I Isolation (독립성)
여러 작업이 동시에 실행돼도 서로 영향을 주지 않아요.
사용자 A ↔ 사용자 B

D Durability (지속성)

성공한 작업은 시스템이 꺼져도
결과가 영구히 저장돼요.



영구 저장

왜 ACID가 필요할까요?



데이터 손상 방지

중간에 오류가 나도
데이터가 깨지지 않아요.



동시 작업 안전

여러 사람이 동시에 사용해도
서로 영향을 주지 않아요.



신뢰할 수 있는 시스템

완료된 작업은 반드시
저장되고 사라지지 않아요.



신생활 비유

계좌 이체를 예로 들면,
출금, 입금, 기록 저장이 모두 성공해야 이체가 완료되고,
하나라도 실패하면 아무 일도 없던 것처럼 되돌려요!



개념 · 참고

: Gonna Need It)

리 만들지 말자!

리 만들면,
져워져요.



한 줄 요약

"지금 필요하지 않은 기능은
만들지 말고,
필요해졌을 때 만들자!"

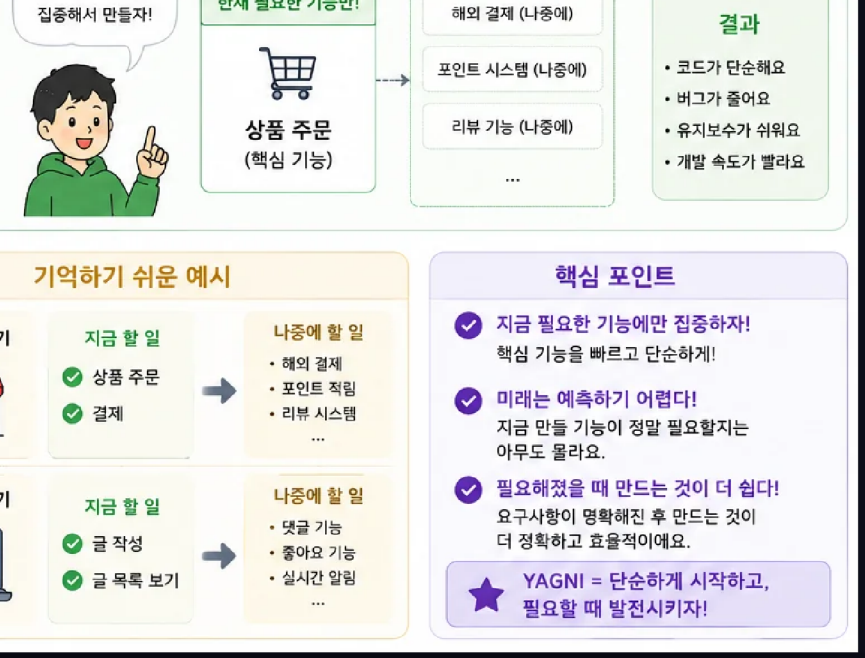
YAGNI를 지킨 경우 (필요할 때 만들기)

지금 필요한 기능만 만들고, 필요해지면 그때 추가해요.

지금은 주문 기능만
필요하니, 이것만

필요해지면 그때!





참고

이렇게 대화

파일을 직접 고친다 → 확인 방법까지.

① **말한다** — 구체적으로

맥락 + 목표 + 제약 + 완료기준

② AI가 계획·수정 — ▸ 는 도구 사용

파일 읽기(Read)·고치기(Edit)

③ 위험한 변경은 물어봄

읽고 ❯ Yes(Enter)로 허락

④ 확인 방법까지 알려줌

직접 눌러보고 → 다음 요청

결과 · ❯ 선택(Enter) · > 내 입력 — 실제 스크린샷 있으면

권고

일 잘 아는 걸로

깅↓ · 빠름 · 잘 모르는 스택은 AI가 부정확한 코드를 내어 직접 수정할 늘어남

se

Vercel

배포

· 단순 CRUD면 백엔드

git push = 자동 배포 · 링크 하나로 폰에서 바로 연다

도 압승

르 · 아니면 그대로

Native/Flutter) · 아니면 웹

FastAPI · 기기기능 꼭이면 그때만 앱 · 원칙은 "AI가 잘

e · 프로그램끼리 데이터를 주고받는 통로

권고

고 바꾸는 법

Opus 4.8

최고 지능 · 어려운 설계·오래 막힌 버그·밤샘 자동 작업

$5 / $25 · 1M

Fable 5

최상위 **추론** · 대부분 과함 — **정말 안 풀릴 때만**

$10 / $50 · 1M

똑똑하고 한도 많이 씀 🧠

수록 한도를 빨리 쓰니 /usage로 잔여 확인 · 모델·가격

Max

eloper

l.com · 010-7320-7733